ST 538 Group 4 Project 2

Paul Anderson, Michael Barton, Kevin Kincaid

This analysis uses the CRIF High Mark vehicle loan dataset to develop a feature set capable of identifying first-EMI default risk across both scored and thin-file borrower segments. The dataset is comprised of 233,154 disbursed loans with a binary default indicator capturing payment failure on the first EMI due date. A notable characteristic of this population is that 56% of borrowers carry no scoreable bureau history, which necessitates treating thin-file and scored segments with care rather than imputing or ignoring the distinction.

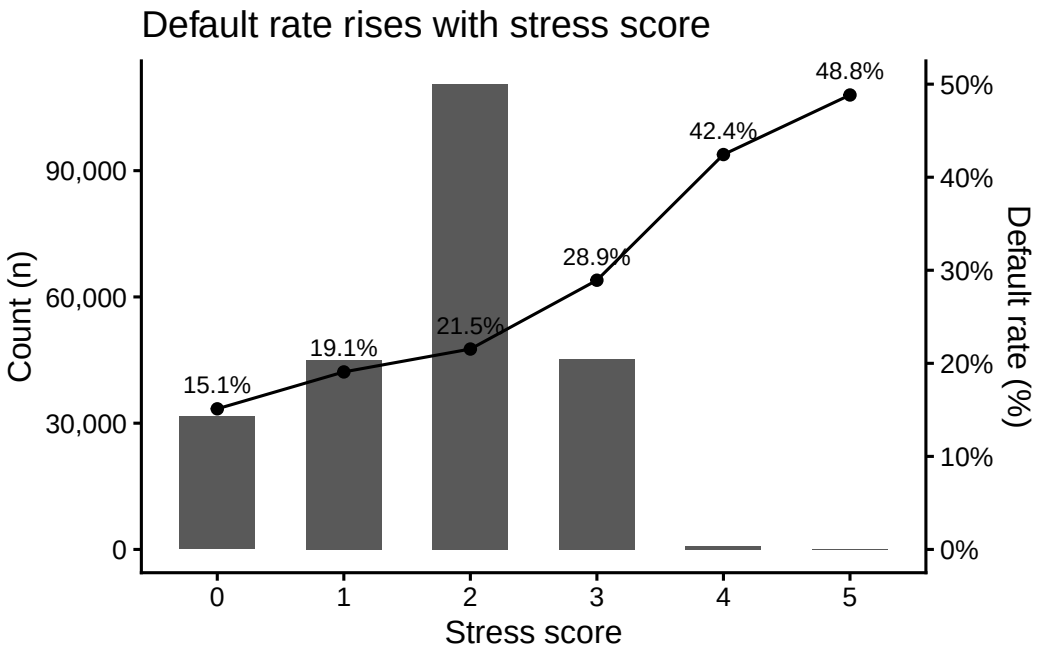
Feature extraction and description

Feature engineering focuses on translating domain knowledge into model-ready variables: loan structure risk (LTV, implied down payment), borrower demographics (age at disbursement, employment type), identity verification depth (KYC document count), credit utilisation and delinquency history across primary and secondary accounts, EMI burden relative to existing obligations, and recent bureau activity including new account uptake and hard enquiries.

A composite stress score aggregating six binary risk flags is also constructed as a diagnostic and interpretability aid. Initial cross-tabulation confirms that the stress score tracks default rate monotonically, rising from 15% at score 0 to 49% at score 5, providing early evidence that the engineered features carry meaningful signal.

Table 1: Default Rate by Stress Score

Stress score	Total (n)	No default	Default	Default rate
0	31,573	26,802	4,771	15.1%
1	44,945	36,371	8,574	19.1%
2	110,556	86,748	23,808	21.5%
3	45,181	32,110	13,071	28.9%
4	813	468	345	42.4%
5	86	44	42	48.8%



Thirty-five percent of loans are high loan to value (LTV) of >80%, combined with a large portion of borrowers having thin credit history (%), it shows a meaningful concentration of risk. The vast majority (94%) of borrowers fall in the prime age band, 13,812 young borrowers (<22 years of age) are worth watching. Fifty-six percent of borrowers are "thin file" meaning they have <12 month of credit history. This makes sense, in India about half of first time car buyers are using lending for the first time. Thin and normal are two different populations, we may want to split the dataset and create a model for thin credit and one for normal, or focus our efforts on normal only–the model will likely suffer trying to work on both.

Stress score is working well as a risk signal. Default raises from 15% at score 0 to ~49% at score 5 (3x). Most of the population sits at 2 (21% default), probably thin-file + high-LTV, but no active delinquncy.The jump from 2 to 3 is steep (21-29%), pri_any_overdue_flag and/or recent_delinquncy_flag are likely doing a lot of work. Scores 4 and 5 are small segments with high default rates (42-49%), it's really up to us if we want to keep them, probably should. Score 0 is not zero (15.1%), even the cleanest borrowers carry default risk. We can use this stress_score as a standalone feature, but it's worth noting that components might actually capture more non-linear interactions than the score itself.

Building Classifiers and Results

After feature extraction and variable selection, we split the dataset into training, validation, and testing subsets using a 70/15/15 split, resulting in approximately 72,000 observations in the training set and 15,000 observations each in the validation and testing sets.

We will utilize two methods to try classifying defaults accurately: Random Forest & XGBoost

Table 2: Random Forest Model Comparison

Model	Dataset	TN	FP	FN	TP	Accuracy	Recall	Precision	Specificity	F1
rf.sim1	Training	57124	395	14256	355	0.797	0.024	0.473	0.993	0.046
rf.sim1	Validation	12415	94	3056	78	0.799	0.025	0.453	0.992	0.047
rf.sim2	Training	57132	387	14246	365	0.797	0.025	0.485	0.993	0.048
rf.sim2	Validation	12407	102	3058	76	0.798	0.024	0.427	0.992	0.046
rf.final	Training	43542	13977	7518	7093	0.702	0.485	0.337	0.757	0.398
rf.final	Validation	9345	3164	1646	1488	0.693	0.475	0.320	0.747	0.382

First, we tried fitting a default random forest classifier to the training data and evaluating its performance on both the training and validation datasets. Due to the strong class imbalance in the loan default variable, the initial models showed poor sensitivity for detecting loan defaults, despite relatively high overall accuracy. We then adjusted model parameters and explored additional tuning strategies, including the use of tuneRF() to optimize the mtry parameter and balanced sampling to improve classification performance. The tuned model substantially improved recall for default predictions, though it also increased the number of false positive classifications. While the final random forest model showed meaningful improvement over the baseline models (Table 2), prediction performance for loan defaults remained moderate, indicating that additional tuning or alternative classification methods may be necessary for highly imbalanced loan data.

Table 3: Confusion Matrix

	0	1
0	11260	1249
1	2895	239

Table 4: XGBoost Peformance by Threshold

Threshold	Accuracy	Recall	Precision	Specificity	F1
0.10	0.2023269	0.9961710	0.2002823	0.0034375	0.3335114
0.15	0.2723263	0.8704531	0.1990514	0.1224718	0.3240097
0.20	0.4600141	0.5178685	0.1896250	0.4455192	0.2776020

Threshold	Accuracy	Recall	Precision	Specificity	F1
0.25	0.6230263	0.2383535	0.1754757	0.7194020	0.2021377
0.30	0.7350892	0.0762604	0.1606183	0.9001519	0.1034184

Next we trained XGBoost classifiers on our cleaned dataset. We did utilize the scale_pos_weight parameter when training the model to try compensating for the imbalance in the response variable. After training the model we found it was doing worse than our tuned forest. In testing the AUC score was 0.5274, confirming the models inaccuracy. We then explored different thresholds and found that 0.10 and 0.15 give better results for the imbalanced data. It may be necessary to utilize LASSO or other methods to try and remove overlapping variables before training the XGBoost models.

Next Steps

Based on our analysis, it is clear that the most pressing issue is to find a method to combat the imbalance present in our data. We may consider utilizing SMOTE or downsampling methods to compensate for this. https://www.geeksforgeeks.org/machine-learning/how-to-use-smote-for-imbalanced-data-in-r/

Appendix

```
# load dependencies
library(tidyverse)
library(lubridate)
library(stringr)
library(knitr)
library(randomForest)

# Week 3 (report 2)
#install.packages("xgboost")
library(xgboost)
library(caret)
library(pROC)
# install.packages("kable")
# library(kable)
# library(kableExtra)
# Project URL https://www.kaggle.com/datasets/avikpaul4u/vehicle-loan-default-prediction?sele

# Load our files (saved manually to data directory)
```

```r
# Training data
train_dat = read.csv('./data/train.csv')

# Testing data (DO NOT LOAD UNTIL WEEK 8 OR 9!!)
# Also, the test.csv does not have Loan_Default, so it may not work for this
#test_dat = read.csv('./data/test.csv')

head(train_dat)
str(train_dat)
# Check for any columns with missing data
sapply(train_dat, anyNA)

# There are no missingness in the data! Pretty sweet!
# helper function - parse tenure string to numeric months
parse_tenure_months <- function(x) {
  years  <- as.numeric(str_extract(x, "\\d+(?=yrs)"))
  months <- as.numeric(str_extract(x, "\\d+(?=mon)"))
  years  <- ifelse(is.na(years),  0, years)
  months <- ifelse(is.na(months), 0, months)
  years * 12 + months
}

# loan terms
train_dat <- train_dat %>%
  mutate(

    # LTV risk flag - regulatory / underwriting threshold
    ltv_high_flag = as.integer(LTV > 80),

    # Implied down payment
    down_payment_amt = ASSET_COST - DISBURSED_AMOUNT,
    down_payment_pct = round((down_payment_amt / ASSET_COST) * 100, 2)

  )

# demographics and disbursement date
train_dat <- train_dat %>%
  mutate(

    # Parse dates - format is "DD-MM-YYYY" per the raw data
    dob_parsed     = dmy(DATE_OF_BIRTH),
    disbursal_date = dmy(DISBURSAL_DATE),
```

```
    # Age at disbursement in whole years
    age_at_disbursal = as.integer(
      interval(dob_parsed, disbursal_date) / years(1)
    ),

    # Age risk segments
    age_segment = case_when(
      age_at_disbursal <  22                          ~ "young_risky",
      age_at_disbursal >= 22 & age_at_disbursal <= 60  ~ "prime",
      age_at_disbursal >  60                          ~ "senior_risky",
      TRUE                                            ~ NA_character_
    ),

    # Disbursement seasonality
    disbursal_month   = month(disbursal_date),
    disbursal_quarter = quarter(disbursal_date),
    disbursal_year    = year(disbursal_date),

    # Indian fiscal year-end flag (Jan-Mar = Q4 of Indian FY)
    fy_end_flag = as.integer(disbursal_month %in% c(1, 2, 3))

  ) %>%
  select(-dob_parsed)   # intermediate parse column no longer needed

# identity
train_dat <- train_dat %>%
  mutate(

    # Total KYC documents submitted (0-6)
    kyc_doc_count = MOBILENO_AVL_FLAG + AADHAR_FLAG + PAN_FLAG +
                    VOTERID_FLAG + DRIVING_FLAG + PASSPORT_FLAG,

    # Strong KYC: has both Aadhaar (biometric) AND PAN (income tax records)
    strong_kyc_flag = as.integer(AADHAR_FLAG == 1 & PAN_FLAG == 1)

  )

# bureau score

# sort(unique(train_dat$PERFORM_CNS_SCORE_DESCRIPTION)) # check the label strings

# CRIF CNS bands A-M run lowest to highest risk.
```

```r
# "Not Scored" has 6 distinct reason codes - all treated as thin-file.
# Ordered worst -> best creditworthiness so ordinal models can use the ranking.

cns_levels <- c(
  # --- Thin-file / unscoreable (placed at bottom = worst/unknown) ---
  "No Bureau History Available",
  "Not Scored: More than 50 active Accounts found",
  "Not Scored: No Activity seen on the customer (Inactive)",
  "Not Scored: No Updates available in last 36 months",
  "Not Scored: Not Enough Info available on the customer",
  "Not Scored: Only a Guarantor",
  "Not Scored: Sufficient History Not Available",
  # --- Scored bands: highest risk first ---
  "M-Very High Risk",
  "L-Very High Risk",
  "K-High Risk",
  "J-High Risk",
  "I-Medium Risk",
  "H-Medium Risk",
  "G-Low Risk",
  "F-Low Risk",
  "E-Low Risk",
  "D-Very Low Risk",
  "C-Very Low Risk",
  "B-Very Low Risk",
  "A-Very Low Risk"
)

# All "Not Scored" reason strings for thin-file flagging
not_scored_reasons <- c(
  "No Bureau History Available",
  "Not Scored: More than 50 active Accounts found",
  "Not Scored: No Activity seen on the customer (Inactive)",
  "Not Scored: No Updates available in last 36 months",
  "Not Scored: Not Enough Info available on the customer",
  "Not Scored: Only a Guarantor",
  "Not Scored: Sufficient History Not Available"
)

train_dat <- train_dat %>%
  mutate(
```

```
    # Ordered factor - preserves risk ranking in tree-based and ordinal models
    cns_score_factor = factor(
      PERFORM_CNS_SCORE_DESCRIPTION,
      levels  = cns_levels,
      ordered = TRUE
    ),

    # Thin-file flag: covers all 7 unscoreable categories
    thin_file_flag = as.integer(
      PERFORM_CNS_SCORE_DESCRIPTION %in% not_scored_reasons
    ),

    # Thin-file reason - retains the specific not-scored reason for analysis
    # (e.g. "Only a Guarantor" may have a different risk profile than "Inactive")
    thin_file_reason = ifelse(
      thin_file_flag == 1,
      PERFORM_CNS_SCORE_DESCRIPTION,
      NA_character_
    ),

    # Clean score: NA out thin-file records so 0 is not treated as a real score
    cns_score_clean = ifelse(thin_file_flag == 1, NA_real_, PERFORM_CNS_SCORE)
    )

# primary accounts
train_dat <- train_dat %>%
  mutate(

    # Active loan utilisation ratio
    pri_active_util_ratio = ifelse(
      PRI_NO_OF_ACCTS > 0,
      round(PRI_ACTIVE_ACCTS / PRI_NO_OF_ACCTS, 4),
      0
    ),

    # Any overdue primary account - binary red flag
    pri_any_overdue_flag = as.integer(PRI_OVERDUE_ACCTS > 0),

    # Incremental leverage: existing balance relative to new loan size
    pri_leverage_ratio = ifelse(
      DISBURSED_AMOUNT > 0,
      round(PRI_CURRENT_BALANCE / DISBURSED_AMOUNT, 4),
```

```
      NA_real_
    ),

    # Average primary loan size historically
    pri_avg_loan_size = ifelse(
      PRI_NO_OF_ACCTS > 0,
      round(PRI_DISBURSED_AMOUNT / PRI_NO_OF_ACCTS, 2),
      0
    ),

    # Sanctioned vs. disbursed gap (large gap may indicate cancellations)
    pri_sanction_disbursed_gap = PRI_SANCTIONED_AMOUNT - PRI_DISBURSED_AMOUNT

  )

# joint accounts
train_dat <- train_dat %>%
  mutate(

    # Binary: any secondary overdue
    sec_any_overdue_flag  = as.integer(SEC_OVERDUE_ACCTS > 0),

    # Has any secondary exposure at all
    sec_has_exposure_flag = as.integer(SEC_NO_OF_ACCTS > 0)

  )

# EMI / debt service
train_dat <- train_dat %>%
  mutate(

    # Total monthly EMI obligation across primary and secondary loans
    total_emi_burden = PRIMARY_INSTAL_AMT + SEC_INSTAL_AMT,

    # New loan EMI relative to existing primary EMI commitment
    # Ratio > 1: new loan eclipses all current EMI obligations combined
    new_loan_emi_ratio = ifelse(
      PRIMARY_INSTAL_AMT > 0,
      round(DISBURSED_AMOUNT / PRIMARY_INSTAL_AMT, 4),
      NA_real_
    )
```

```
  )

# recent bureau activity
train_dat <- train_dat %>%
  mutate(

    # Binary: any delinquency in the last 6 months
    recent_delinquency_flag = as.integer(DELINQUENT_ACCTS_IN_LAST_SIX_MONTHS > 0),

    # Inquiry-to-new-account ratio
    # When NEW_ACCTS = 0: return raw inquiry count - all enquiries were rejected
    inquiry_to_new_acct_ratio = ifelse(
      NEW_ACCTS_IN_LAST_SIX_MONTHS > 0,
      round(NO_OF_INQUIRIES / NEW_ACCTS_IN_LAST_SIX_MONTHS, 4),
      NO_OF_INQUIRIES
    ),

    # Parse "Xyrs Ymon" tenure strings to numeric months
    avg_acct_age_months   = parse_tenure_months(AVERAGE_ACCT_AGE),
    credit_history_months = parse_tenure_months(CREDIT_HISTORY_LENGTH),

    # Thin credit history flag (<12 months = thin-file risk)
    short_history_flag = as.integer(credit_history_months < 12)

  )

# composite financial stress score
train_dat <- train_dat %>%
  mutate(

    # Additive count of binary risk flags
    # Useful for EDA, stakeholder comms, and model sanity checks
    stress_score = pri_any_overdue_flag    +
                   sec_any_overdue_flag    +
                   recent_delinquency_flag +
                   ltv_high_flag           +
                   short_history_flag      +
                   thin_file_flag,

    # High-stress flag: 3 or more simultaneous risk indicators
    high_stress_flag = as.integer(stress_score >= 3)
```

```
  )

# employment type na results fix
train_dat <- train_dat %>%
    mutate(EMPLOYMENT_TYPE = na_if(EMPLOYMENT_TYPE, ""))
# validation
cat("Rows:", nrow(train_dat), "\n")
cat("Total columns after engineering:", ncol(train_dat), "\n")
cat("Net new columns:", ncol(train_dat) - 41, "\n\n")

cat("--- LTV high flag (>80%) ---\n")
print(table(train_dat$ltv_high_flag, useNA = "ifany"))

cat("\n--- Age segment ---\n")
print(table(train_dat$age_segment, useNA = "ifany"))

cat("\n--- Employment type ---\n")
print(table(train_dat$EMPLOYMENT_TYPE, useNA = "ifany"))

cat("\n--- CNS score factor levels (confirm all levels matched) ---\n")
print(levels(train_dat$cns_score_factor))
cat("Unmatched descriptions (NAs in factor):",
    sum(is.na(train_dat$cns_score_factor)), "\n")

cat("\n--- Thin-file borrowers ---\n")
print(table(train_dat$thin_file_flag, useNA = "ifany"))

cat("\n--- Primary any overdue ---\n")
print(table(train_dat$pri_any_overdue_flag, useNA = "ifany"))

cat("\n--- Recent delinquency flag ---\n")
print(table(train_dat$recent_delinquency_flag, useNA = "ifany"))

cat("\n--- Stress score distribution ---\n")
print(table(train_dat$stress_score, useNA = "ifany"))

cat("\n--- avg_acct_age_months (first 10 rows) ---\n")
print(data.frame(
  raw    = head(train_dat$AVERAGE_ACCT_AGE, 10),
  parsed = head(train_dat$avg_acct_age_months, 10)
))
```

```r
cat("\n--- credit_history_months (first 10 rows) ---\n")
print(data.frame(
  raw    = head(train_dat$CREDIT_HISTORY_LENGTH, 10),
  parsed = head(train_dat$credit_history_months, 10)
))
# default rate by stress score table
stress_table <- train_dat %>%
  group_by(stress_score) %>%
  summarise(
    n           = n(),
    n_default   = sum(LOAN_DEFAULT, na.rm = TRUE),
    .groups     = "drop"
  ) %>%
  mutate(
    n_no_default  = n - n_default,
    default_rate  = paste0(round(n_default / n * 100, 1), "%")
  ) %>%
  select(
    `Stress score`  = stress_score,
    `Total (n)`     = n,
    `No default`    = n_no_default,
    `Default`       = n_default,
    `Default rate`  = default_rate
  )

knitr::kable(
  stress_table,
  format      = "simple",
  caption     = "Default Rate by Stress Score",
  align       = c("c", "r", "r", "r", "r"),
  format.args = list(big.mark = ",")
)
# default rate by stress score graphically
stress_summary <- train_dat %>%
  group_by(stress_score) %>%
  summarise(
    n            = n(),
    n_default    = sum(LOAN_DEFAULT, na.rm = TRUE),
    .groups      = "drop"
  ) %>%
  mutate(default_rate = n_default / n * 100)
```

```r
scale_factor <- max(stress_summary$n) / 50

ggplot(stress_summary, aes(x = factor(stress_score))) +

  geom_col(aes(y = n), width = 0.6) +

  geom_line(aes(y = default_rate * scale_factor, group = 1)) +
  geom_point(aes(y = default_rate * scale_factor)) +

  geom_text(
    aes(y = default_rate * scale_factor, label = paste0(round(default_rate, 1), "%")),
    vjust = -0.9, size = 3.2
  ) +

  scale_y_continuous(
    name     = "Count (n)",
    labels   = scales::comma,
    sec.axis = sec_axis(~ . / scale_factor, name = "Default rate (%)",
                        labels = scales::label_number(suffix = "%"))
  ) +

  theme_classic(base_size = 12) +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor   = element_blank()) +

  labs(
    title = "Default rate rises with stress score",
    x     = "Stress score"
  )
# engineered feature index table
feature_index <- tribble(
  ~Category,          ~Feature,
  "Loan terms",       "ltv_high_flag",
  "Loan terms",       "down_payment_amt",
  "Loan terms",       "down_payment_pct",
  "Demographics",     "age_at_disbursal",
  "Demographics",     "age_segment",
  "Demographics",     "disbursal_month",
  "Demographics",     "disbursal_quarter",
  "Demographics",     "disbursal_year",
  "Demographics",     "fy_end_flag",
  "KYC",              "kyc_doc_count",
```

```
  "KYC",               "strong_kyc_flag",
  "Bureau score",      "cns_score_factor",
  "Bureau score",      "thin_file_flag",
  "Bureau score",      "thin_file_reason",
  "Bureau score",      "cns_score_clean",
  "Primary credit",    "pri_active_util_ratio",
  "Primary credit",    "pri_any_overdue_flag",
  "Primary credit",    "pri_leverage_ratio",
  "Primary credit",    "pri_avg_loan_size",
  "Primary credit",    "pri_sanction_disbursed_gap",
  "Secondary credit", "sec_any_overdue_flag",
  "Secondary credit", "sec_has_exposure_flag",
  "EMI / DSCR",        "total_emi_burden",
  "EMI / DSCR",        "new_loan_emi_ratio",
  "Recent activity",   "recent_delinquency_flag",
  "Recent activity",   "inquiry_to_new_acct_ratio",
  "Recent activity",   "avg_acct_age_months",
  "Recent activity",   "credit_history_months",
  "Recent activity",   "short_history_flag",
  "Composite",         "stress_score",
  "Composite",         "high_stress_flag"
)

# collapse each category's features into a single comma-separated string
feature_index_wide <- feature_index %>%
  group_by(Category) %>%
  summarise(Features = paste(Feature, collapse = ", "), .groups = "drop")

knitr::kable(feature_index_wide, format = "simple", col.names = c("Category", "Features"),
             caption = "Feature Index Table")
train_dat_normal <- train_dat %>%
  filter(thin_file_flag == 0)

train_dat_thin <- train_dat %>%
  filter(thin_file_flag == 1)

cbind(count(train_dat_normal),count(train_dat_thin))
set.seed(2026)

# Set our splits
train_percent <- 0.7
not_train_percent <- 0.3
```

```r
# not_train split up
validate_percent <- 0.5 # Ends up being 0.15
test_percent <- 0.5 # Ends up being 0.15


# splitting into train/not_train -   this had an error with nrow(train_dat) producing too lor
sample <- sample(c(TRUE, FALSE), nrow(train_dat_normal), replace=TRUE, prob=c(train_percent,r
training  <- train_dat_normal[sample, ]



not_training   <- train_dat_normal[!sample, ]

# splitting into validation/test
train_sample <- sample(c(TRUE, FALSE), nrow(not_training), replace=TRUE, prob=c(validate_perc
validation  <- not_training[train_sample, ]
testing   <- not_training[!train_sample, ]


rbind(count(training), count(validation), count(testing))

# # Did they default?
# train_dat |>
#   summarise(
#     defaulted_yes_overall = sum(LOAN_DEFAULT == 1),
#     defaulted_no_overall = sum(LOAN_DEFAULT == 0)
#
#   )
#
# train |>
#   summarise(
#     defaulted_yes_train = sum(LOAN_DEFAULT == 1),
#     defaulted_no_train = sum(LOAN_DEFAULT == 0)
#   )
#
# test |>
#   summarise(
#     defaulted_yes_test = sum(LOAN_DEFAULT == 1),
#     defaulted_no_test = sum(LOAN_DEFAULT == 0)
#
#   )
  #sapply(testing[,c(-10, -56, -66, -41)], anyNA)
```

```
  head(training[,c(-1, -5, -7, -10, -11, -12, -13, -14, -17, -18, -19, -26, -27, -29, -31, -3

 #which(names(training) %in% c("new_loan_emi_ratio","STATE_ID"))
  # the following have NA values which do not work with forest
  #10 EMPLOYMENT_TYPE
  #56 thin_file_reason
  #66 new_loan_emi_ratio

  #41 is predictor LOAN_DEFAULT
  sapply(training,anyNA)
  #sapply(training[,c(-10, -56, -66, -42)], anyNA)

  #training |> filter(is.na(thin_file_reason))
  #training |> filter(is.na(high_stress_flag))

drop_cols <- c(-1, -5, -7, -10, -11, -12, -13,
               -14, -17, -18, -19, -26, -27,
               -29, -31, -32, -33, -35,
               -56, -66, -41)

# Save the filtered data into variables to make it easier
x_training <- training[, drop_cols]
x_validation <- validation[, drop_cols]
  # rf.sim1 = randomForest(training[,c(-1, -5, -7, -10, -11, -12, -13, -14, -17, -18, -19, -
  #                        factor(training$LOAN_DEFAULT))

  # pred.rf.sim1 = rf.sim1$predicted
  # table(training[,41], pred.rf.sim1)
  #
  # varImpPlot(rf.sim1)
  #

  # #training |> filter(high_stress_flag == 1)
  #
  # #    pred.rf.sim1
  # #       0     1
  # # 0 57124   395
  # # 1 14256   355
# pred.rf.sim1 <- predict(
#   rf.sim1,
#   newdata = x_validation,
#   type = "class"
```

```
# )
#
# table(validation[,41], pred.rf.sim1)
#
#   #  pred.rf.sim1
#   #       0     1
#   # 0 12415    94
#   # 1  3056    78
# rf.sim2 <- randomForest(
#   x = training[, c(-1, -5, -7, -10, -11, -12, -13,
#                    -14, -17, -18, -19, -26, -27,
#                    -29, -31, -32, -33, -35,
#                    -56, -66, -41)],
#
#   y = factor(training$LOAN_DEFAULT),
#
#   ntree = 500,
#   importance = TRUE
# )
#
# pred.rf.sim2 <- rf.sim2$predicted
# table(training[,41], pred.rf.sim2)
# # pred.rf.sim2
# #         0     1
# #   0 57132   387
# #   1 14246   365
#
# pred.rf.sim2 <- predict(
#   rf.sim2,
#   newdata = x_validation,
#   type = "class"
# )
#
#
# table(validation[,41], pred.rf.sim2)
#
#
#   #  pred.rf.sim2
#   #       0     1
#   # 0 12407   102
#   # 1  3058    76
# CURRENTLY TOO LARGE!!!
```

```
# Random Forest Classifier for Digit Recognition Data
  #rf.digit = randomForest(training[,c(-10, -56, -66, -41)], factor(training$LOAN_DEFAULT),
  # Predicted response labels
  #rfPred.digit = (rf.digit$test)$predicted
  # Misclassification errors in the test data
 # table(digit_no.test, rfPred.digit)
  # Variable Importance Plot
  #varImpPlot(rf.digit)


# https://www.statology.org/random-forest-in-r/
# Commented out to allow the doc to render faster
# tune.rf <- tuneRF(
#   x = training[, drop_cols],
#
#   y = factor(training$LOAN_DEFAULT),
#
#   stepFactor = 1.5,
#   improve = 0.01,
#   ntreeTry = 300
# )
# Commented out to allow the doc to render faster
# best.mtry <- tune.rf[which.min(tune.rf[,2]),1]
#
# rf.final <- randomForest(
#   x = training[, drop_cols],
#   y = factor(training$LOAN_DEFAULT),
#
#   ntree = 1000,
#   mtry = best.mtry,
#
#   importance = TRUE,
#
#   sampsize = c(10000, 10000)
# )
# # Commented out to allow the doc to render faster
#   pred.rf.final = rf.final$predicted
#   table(training[,41], pred.rf.final)
#   # pred.rf.final
#   #       0     1
#   # 0 43542 13977
#   # 1  7518  7093
```

```
#   pred.rf.final <- predict(
#   rf.final,
#   newdata = x_validation,
#   type = "class"
# )
#
#
# table(validation[,41], pred.rf.final)
#
#   # pred.rf.final
#   #      0    1
#   # 0 9345 3164
#   # 1 1646 1488
# Pulling everything from the commented out models above.
rf_results <- tribble(
  ~Model, ~Dataset, ~TN, ~FP, ~FN, ~TP,

  "rf.sim1", "Training",   57124, 395, 14256, 355,
  "rf.sim1", "Validation", 12415, 94, 3056, 78,

  "rf.sim2", "Training",   57132, 387, 14246, 365,
  "rf.sim2", "Validation", 12407, 102, 3058, 76,

  "rf.final", "Training",  43542, 13977, 7518, 7093,
  "rf.final", "Validation", 9345, 3164, 1646, 1488
) %>%
  mutate(
    Accuracy = (TP + TN) / (TP + TN + FP + FN),

    Recall = TP / (TP + FN),

    Precision = TP / (TP + FP),

    Specificity = TN / (TN + FP),

    F1 = 2 * (Precision * Recall) / (Precision + Recall)
  )

kable(
  rf_results %>%
    mutate(
      across(
```

```
        c(Accuracy, Recall, Precision, Specificity, F1),
        round,
        3
      )
    ),
  caption = "Random Forest Model Comparison"
)
# Let's try going back to LASSO, see which features we get
# XG Boost

# convert to matrices
x_train <- model.matrix(
  ~ . - 1,
  data = training[, drop_cols]
)

x_valid <- model.matrix(
  ~ . - 1,
  data = validation[, drop_cols]
)

y_train <- training$LOAN_DEFAULT
y_valid <- validation$LOAN_DEFAULT



# Deal with the imbalance
scale_pos_weight <- sum(y_train == 0) / sum(y_train == 1)

scale_pos_weight


# Training matrices
dtrain <- xgb.DMatrix(
  data = x_train,
  label = y_train
)

dvalid <- xgb.DMatrix(
  data = x_valid,
  label = y_valid
)
```

```r
# Initial XGBoost model
params <- list(
  objective = "binary:logistic",
  eval_metric = "auc",
  max_depth = 6,
  eta = 0.05,
  subsample = 0.8,
  colsample_bytree = 0.8,
  scale_pos_weight = scale_pos_weight
)

xgb_model <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 300,
  watchlist = list(train = dtrain, validation = dvalid),
  verbose = 1
)


# Get predictions
pred_prob <- predict(xgb_model, dvalid)

pred_class <- ifelse(pred_prob > 0.30, 1, 0)

conf_mat <- table(
  Actual = y_valid,
  Predicted = pred_class
)

kable(as.data.frame.matrix(conf_mat),
      caption = "Confusion Matrix")
roc_obj <- roc(y_valid, pred_prob)

auc(roc_obj)
thresholds <- c(0.10, 0.15, 0.20, 0.25, 0.30)

xgb_threshold_results <- lapply(thresholds, function(t) {
  pred_class <- ifelse(pred_prob > t, 1, 0)
  tab <- table(
    Actual = y_valid,
    Predicted = pred_class
```

```
  )

  TN <- tab["0", "0"]
  FP <- tab["0", "1"]
  FN <- tab["1", "0"]
  TP <- tab["1", "1"]

  data.frame(
    Threshold = t,
    Accuracy = (TP + TN) / sum(tab),
    Recall = TP / (TP + FN),
    Precision = TP / (TP + FP),
    Specificity = TN / (TN + FP),
    F1 = 2 * ((TP / (TP + FP)) * (TP / (TP + FN))) /
      ((TP / (TP + FP)) + (TP / (TP + FN)))
  )
}) |>
  bind_rows()

knitr::kable(
  xgb_threshold_results,
  format      = "simple",
  align       = c("c", "r", "r", "r", "r", "r"),
  caption     = "XGBoost Peformance by Threshold"
)
```